Contents

Known bugs and v1.0 limitations

v1.0 known limitations (workarounds documented; v1.1 candidates)

These are not bugs in the strict sense — they're scope decisions for v1.0 that we may revisit based on real-world testing.

`.dta` round-trip loses `xtset` state

`save mydata.dta` then `use mydata.dta` preserves variable types, formats, variable labels, and value labels, but does **not** preserve the `xtset` panel/time variable assignment. Stata writes this as a dataset characteristic (`_dta[iis]`, `_dta[tis]`); readstat doesn't expose characteristic writing in its public API, and hand-writing characteristic blocks in raw .dta format is a v1.1 task.

Workaround: re-run `xtset country year` after `use`. This is the standard Stata-do-file convention anyway, so most users won't notice.

ARIMA uses conditional likelihood, not exact ML (no Kalman filter)

tea's `arima` command computes parameters by minimizing the SSR of the recursively-computed residuals, conditioning on ε_{<0}=0 (the conditional likelihood approach). This gives consistent estimates but slightly different finite-sample behavior than Stata's default exact ML via the Kalman filter:

- Absolute log-likelihood values differ from Stata's (we drop the initial-state contribution)
- AR coefficients near the unit-root boundary may converge less robustly
- No seasonal ARIMA (sar, sma) in v1.0

Workaround: for serious time-series work, escape to R's `arima()` or Python's `statsmodels.tsa.arima`.

Factor variable composition with TS ops

`i.country#c.L.gdp` (factor interaction with a lagged regressor) is **not** supported directly. The factor module operates on column names, not on materialized TS-op temps.

Workaround: pre-compute the lag manually: `gen L1_gdp = L.gdp`, then use `i.country#c.L1_gdp`.

`xtreg, be` doesn't weight by T_i

Stata's default `xtreg, be` uses a slight variance correction for unbalanced panels (down-weights groups with few observations). tea uses straight OLS on the panel means. For balanced panels the answers are identical. For unbalanced panels they can differ by a few percent in the SEs.

MA(1) and ARMA SE precision

ARIMA standard errors are computed from a numerical-differentiation Hessian (forward differences scaled by 1e-5). For models with q>0 (MA components), the SEs can be off by 1-5% compared to analytical or exact-ML Hessians. Point estimates are unaffected.

Variable-name length limit (32 chars)

A residual Stata-12 limit. Factor-variable interaction names like `2010.year#1985.country#c.gdp` (28 chars) fit; longer 3-way interactions truncate. In practice not a problem.

Graphics: covered since v1.6.6

`scatter` / `line` / `histogram` (v1.3), plus multi-series `twoway` with lowess, `graph box` with two-level `over()`, and the name() registry with `graph combine` (v1.6.6). Remaining gaps: no `by()` faceting, no bar/pie, no graph editor.

CHANGELOG (recently fixed)

Recently fixed

Tier 4 — econometric completion (NEW for v1.0)

`ivregress 2sls` (in `src/regress.c` ~300 lines):

Syntax: `ivregress 2sls y [exog] (endog = instruments) [, vce(robust|cluster v)]`. Full first-stage regression of each endogenous regressor on the combined instrument matrix W = [exog, Z, _cons] via dgels, second-stage $\beta=(\hat{X}'\hat{X})^{-1}\hat{X}'y$; residuals computed using ORIGINAL X (not fitted) for SE; sandwich V = A·B·A for robust/cluster ($A=(\hat{X}'\hat{X})^{-1}$, B uses original-X residuals). HC1 / CR1 finite-sample adjustments. First-stage F-stat reported (minimum across endogenous regressors — the weak-instrument worry case).

Verified on textbook endogeneity case: with cov(x, u) ≠ 0, OLS biased to 2.49; IV recovers 1.997 against true 2.0, first-stage F=1364.

`poisson` — count outcomes via the existing MLE driver:

Added `mle_family_poisson` to src/mle.c with `poisson_per_obs`: μ=exp(η) clipped at η<700; score=y-μ; weight=μ; loglik=yη-μ (drops the log(y!) constant — absolute loglik differs from

Stata but β identical). Added `count_validate` (rejects y<0). Generalized do_glm_binary header text to "Poisson regression". Verified intercept-only: β_cons=log(ȳ)=0.6931 for ȳ=2 (exact).

`xtreg, be` (between estimator):

Replaced the "not implemented" stub. After group-mean computation (already done in the FE/RE path), branch into BE: build G_obs × (K_orig+1) matrix Xb (panel means + appended 1s column for cons), OLS via dgels, σ^2_be=RSS/(G-K), $V=\sigma^2\cdot(\bar{X}'\bar{X})^{-1}$ via Cholesky. Verified hand-checkable case: panel data with $\bar{y}_i = 5 + 2\bar{x}_i$ recovers β_x=2, _cons=5, R^2_b=1.0000 exactly.

`arima` (new module `src/arima.c`, ~580 lines):

Conditional-likelihood ARIMA(p,d,q) with optional ARIMAX exog regressors. Syntax: `arima y [exog], arima(p d q) [noconstant]`. Algorithm: - Difference y in place d times - Pack params as [μ, β_K_ex, φ_p, θ_q] - Recursive residual computation with ε_{<0}=0 conditional assumption - Gauss-Newton iteration with step halving (max 50 iterations) - SSR via finite-difference gradient + Hessian - $V \approx 2\sigma^2\cdot\text{Hess}^{-1}$ via dgetrf/dgetri with ridge

Verified: AR(1) on strong-signal data recovers φ=0.7010 (true 0.7). AR(1) on US WEO growth data matches OLS lag regression to 3 digits ($\hat{\varphi}$ = 0.154 both). MA(1) Monte Carlo with rnormal() noise: θ=0.429 (true 0.5) at N=500.

`runiform()` and `rnormal()` (in eval.c):

PCG32-based PRNG with Box-Muller normal. `set seed N` for reproducibility. Essential for Monte Carlo, simulation, and bootstrap workflows. Tested against expected distributional moments: mean 0.492, sd 0.291 for u (vs true 0.5, 0.289); mean 0.001, sd 1.015 for z (vs true 0, 1).

Tier 3 — output and reproducibility (NEW)

`estimates` command suite (new module `src/estcmd.c`):

```
estimates store  NAME      clone last_est, save under name
estimates restore NAME     load NAME → last_est (for predict/margins/test)
estimates dir              list saved with cmd, N, depvar
estimates drop NAME...     remove named entries; `_all` clears
estimates table NAMES      compact side-by-side comparison table
                           (options: se, t, p, star, stats(N r2 rmse F ...))
```

Aliased as `est`. Workspace gains a linked-list field `stored_est` (in `dataset.h`); freed by `ws_free`. Built on the existing `est_clone` infrastructure from the hausman work.

`estout` — publication-ready tables (new module `src/estout.c`):

```
estout [names] [using FILE] [, format(latex|markdown|md|plain)
                               se|t|p stars stats(...) nogaps
                               title("...") label(tab:foo)
                               keep(varlist) drop(varlist)
                               nomtitles mtitles("M1" "M2" ...)]
```

Default format is LaTeX (per the IMF / academic-economics convention). LaTeX output produces a `tabular` block with significance stars in the superscript style (`$^{***}$`), variable names properly escaped, and a footnote legend. Stata-style `\_cons` for the constant term. Markdown and plain text variants also supported.

Verified end-to-end on a three-model comparison with the standard options: stars apply z-distribution thresholds (10/5/1%), SEs go in parentheses below each coef, stats footer rows handle N/r2/r2_a/rmse/F/ df_r/df_m/sigma_u/sigma_e/rho.

`log` audit and fix:

The `log` command captures output via a printf-macro override in `commands.c`. Problem: `display` lives in `interp.c` and was using bare `printf`, so its output went to stdout but not the log file. Fixed by making `g_logfp` extern and adding a manual tee from `do_display`. Round-trip verified: `log using file.log` now captures both command echoes (`. display ...`) AND their output (`_b[x] = 2`).

`.dta` round-trip audit:

Spot-checked save → use preservation: - Variable types: □ - Variable formats (`%8.2f`, `%td`): □ - Variable labels (`label variable x "GDP per capita"`): □ - Value labels (`label define`, `label values`): □ - Numeric values: □ (down to floating-point precision) - String columns: □

Not preserved in `.dta`: `xtset` panel/time variable assignment. Stata writes this as a dataset characteristic (`_dta[iis]`, `_dta[tis]`); readstat doesn't expose characteristic writing in its public API, and hand-writing the characteristic block in raw .dta format is a v1.1 project. Workaround for v1.0: re-run `xtset country year` after `use`, which is also the standard Stata-do-file convention.

Tier 2 — factor variables: i., c., #, ##, ib. (NEW)

The Stata factor-variable grammar, now supported across every estimator that goes through `tsop_expand_varlist` (regress, xtreg fe/re, logit, probit, predict, margins). Lives in new module `src/factor.[ch]`.

Grammar:

```
i.var           dummies for each non-base level (base = smallest by default)
ib<n>.var       same, but base = level n
c.var           explicit continuous (display prefix; mostly for use in #)
A#B             interaction: cross product of A's and B's columns
A##B            main effects + interaction: equivalent to "A B A#B"
```

Examples (assume country has levels {1,2,3}, year has {2010,2011,2012}):

```
i.country           → 2 cols: "2.country" "3.country"
ib2.country         → 2 cols: "1.country" "3.country"
i.country#c.gdp     → 2 cols: "2.country#c.gdp" "3.country#c.gdp"
i.country#i.year    → 4 cols: "2.country#2011.year" "2.country#2012.year"
                              "3.country#2011.year" "3.country#2012.year"
i.country##c.gdp    → 5 cols total: 2.country, 3.country, gdp,
                                    2.country#c.gdp, 3.country#c.gdp
```

Architecture:

- `factor.c` is a sibling to `tsop.c`: tokens are parsed into atoms (kind 'I' for `i.`, 'C' for `c.`, 'L' for coef-name "2.x", or bare variable in a # context). Atoms join into terms via `#`. `##` is unrolled to the power-set of single-term products (main effects + all interactions).
- `factor_is_factor_token` is the dispatch test; `factor_expand_token` produces a snapshot-able list of temp frame columns.

- Plumbed into `tsop_expand_varlist`: each token is tried as a TS-op first; if that returns "not mine," tried as a factor-var; if that returns "not mine," falls through to plain `varlist_expand`.

Predict/margins re-materialisation (the subtle bit):

When `predict` sees a coefficient named `2.country` or `2.country#c.gdp`, the column doesn't exist anymore — it was a temp dropped after the estimator finished. Solution: treat coef-name form `<level>.<varname>` as a new atom kind 'L' (level-pinned indicator). When `tsop_expand_varlist` is called with `"2.country"`, it dispatches to factor, which produces a single column with values 1{country==2}. Same mechanism handles `2.country#c.gdp` (indicator × continuous).

Verified:

Spec	Hand-checkable result
`regress y x i.country` on y = α_c + x	β=1, _cons=α_1, 2.country=α_2-α_1, …
`regress y x ib2.country`	_cons=α_2, 1.country=α_1-α_2
`regress y c.x#c.x` on y = x^2	coef = 1.0 exactly
`regress y i.country##c.x` on y = α_c + β_c·x	reproduces all α and β exactly
`regress growth L.growth i.cid` (LSDV) on WEO	matches `xtreg fe` exactly

`predict` after `logit y x i.g` now works (was the open bug from the previous session). `margins, dydx(*)` after factor-var estimation works.

Restrictions in v1.0:

- `i.var` requires `var` to be numeric. Use `encode` for string ids.
- Composition with TS-ops (e.g., `i.country#c.L.gdp`) is not supported. Manually pre-compute the lag (`gen L1_gdp = L.gdp`) and use `c.L1_gdp`.
- Cell-count cap of 10000 columns per token (huge interactions blocked).
- Variable-name length limit of 32 chars means very long interaction names truncate. In practice not a problem for typical use.

Tier 1 polish — statistical functions, egen extensions, _b/_se, formats

A pass over the everyday-use surface in preparation for v1.0. An audit showed that tea's expression-language function library was already very comprehensive (math, trig, hyperbolic, strings, dates, distributions, glob+regex, value/variable labels, encode/decode/destring/tostring, recode, format). The gaps that mattered:

New statistical functions (in `src/eval.c`): - `normalden(x)` / `normalden(x,σ)` / `normalden(x,μ,σ)` — normal PDF - `lnnormal(x)` — log Φ(x), stable for large negative x via the asymptotic expansion (reuses `tea_log_normal_cdf`) - `lnnormalden(x)` — log φ(x) closed-form

Better regex (in `src/eval.c`): - `regexs(n)` — extract n-th submatch from the last `regexm` call. Uses a file-scope `tea_regex_submatch[20]` buffer. Index 0 is the whole match; 1..N are capture groups. - `regexr(s, pat, rep)` — replace first regex match with a string

Extended egen (in `src/commands.c`): - New aggregators: `median`, `iqr`, `p25`, `p75`, `p50`, `pc` (with option `p(N)` to specify percentile), `rank`, `tag` - Multi-argument row functions now work: `rowtotal(v1 v2 v3)`, `rowmean`, `rowmin`, `rowmax`, `rowsd`, `rowmiss`, `rownonmiss` (the previous

implementation was single-arg only) - `tag(g1 g2)` marks the first row of each group as 1, others 0 - `rank(x)` uses average-rank-for-ties

Format application (in `src/commands.c::fmt_cell`): Custom printf-style numeric formats set via `format var %w.dF` are now honored by `list` (previously only date formats like `%td` were).

Coefficient/SE macros after estimation (in `src/regress.c` + `src/interp.c`):

`_b[name]` and `_se[name]` are now accessible everywhere `e(N)` is. The estimators (regress, xtreg, logit, probit) store macros keyed by `_b[varname]` and `_se[varname]` (including bracketed names like `_b[L.growth]` for TS-op regressors). `macro_expand` recognises the pattern and substitutes outside double quotes; inside quotes the text stays literal. Standard Stata behaviour.

This unlocks idioms like:

```
quietly regress y x
display "t-stat = " _b[x] / _se[x]
gen yhat = _b[_cons] + _b[x]*x
```

Bug fix — quoted-string macro substitution (in `src/interp.c`): `display "e(N) = " e(N)` used to produce `"3 = 3"` because the macro expander substituted both occurrences of `e(N)`, including the one inside the quoted string. Fixed by tracking `in_dquote` state and skipping `e()` / `r()` / `_b[]` / `_se[]` substitution inside strings. Stata's behaviour preserved: backtick-locals and `$globals` are still substituted inside strings, since those are Stata's quote-aware substitution syntax.

`margins` — marginal effects after regress / logit / probit (NEW)

```
margins, dydx(*)              AME for all continuous regressors
margins, dydx(x1 x2)          AME for specified regressors
margins, dydx(*) atmeans      MEM (effects at sample means)
margins, dydx(x1)             AME for x1 only
```

After OLS, the AME of x_k is exactly β_k and the SE is exactly SE(β_k) — the margins output reproduces the coefficient table. Useful as a sanity check.

After logit / probit, AMEs are the economically meaningful number: how the predicted *probability* changes with x, averaged across the sample. Coefficients themselves aren't directly interpretable (they're log-odds for logit, latent-index units for probit).

Math:

Logit: m_k = (1/N) Σ_i Λ(X_iβ)·(1-Λ(X_iβ)) · β_k Probit: m_k = (1/N) Σ_i φ(X_iβ) · β_k OLS: m_k = β_k

Delta-method SE: V(m_k) = G_k · V(β) · G_k', where G_k is the K-vector ∂m_k/∂β_j = (1/N) Σ_i [g'(η_i) X_ij β_k + g(η_i) 1{j=k}]. Uses the full V(β) matrix from the previous estimation (so robust/cluster SEs flow through automatically).

Verified: - Logit and probit on the same data produce nearly identical AMEs (a known result — they only differ at the tails, which contribute little to the average). In one test case AME(logit, x1) = 0.12237, AME(probit, x1) = 0.12253 — different in the fifth decimal. - AME for OLS reproduces the regress coefficient table exactly. - TS-op coefficient names (`L.x` etc.) work via the same snapshot-then-drop trick `predict` uses.

Deferred for v2: - `margins x1` (predicted outcomes at varying x1 — useful for plots) - `margins, at(x1=(0 1) x2=10)` (predictions at specified values) - Discrete-change effects for

indicator variables - Marginal effects after xtreg (would need within-effect-aware formulas)

`predict` extended to logit, probit, and xtreg (NEW)

The old `predict` worked only after regress, with options `xb` and `residuals`. Rewritten to dispatch on the previous estimator's `cmd`:

```
After regress:
  predict yhat            xb (default)
  predict r, residuals    y - Xβ
  predict s, stdp         standard error of the linear prediction

After logit / probit:
  predict p               predicted probability (default)
  predict idx, xb         linear index Xβ

After xtreg (fe or re):
  predict yhat            xb (default; does NOT include u_i)
  predict u, u            α̂i  (panel effect)
  predict e, e            idiosyncratic residual y - Xβ - α̂i
  predict ue, ue          α̂i + e  (= y - Xβ)
  predict xbu, xbu        Xβ + α̂i  (in-sample prediction)
```

The dispatch lives in `predict_resolve_kind` and the per-row evaluation is one big switch. Sensible errors when options don't match the preceding command (e.g. `predict p, pr` after regress fails cleanly).

TS-op coefficient names: when `e->xnames[j]` is `L.growth` (TS-op form), the column doesn't exist in the frame anymore at predict time, but we can re-materialize it via the shared `tsop_expand_varlist`. A subtlety: we have to **snapshot** the materialized data into our own buffer, drop the temps, THEN add the new prediction variable — otherwise `tsop_drop_temps` would drop the just-added prediction column instead of the temp. This is what an earlier draft got wrong (a regression test would have shown a list-not-found error).

xtreg specifics: $\hat{\alpha}_i$ is computed in one pass over the in-sample rows (`e->used` tells us which) by streaming through the panel-sorted data and accumulating $\bar{y} - \bar{x}'\beta$ per panel. For predictions on rows that weren't in the estimation sample, $\hat{\alpha}$ is unknown so `u/e/ue/xbu` return missing.

Verified on the hand-checkable FE case (3 panels × 4 obs, perfect fit) where the math is exact: β=1, α=10/20/30, e=0, xbu=y. Also tested on WEO AR(1) growth where predict reproduces the fit yhat + r = growth exactly.

`logit` and `probit` — binary-outcome MLE (NEW)

```
logit  y x1 x2 ... [if] [in] [weight], [vce(robust|cluster v)]
probit y x1 x2 ... [if] [in] [weight], [vce(robust|cluster v)]
```

Implemented via a Newton-Raphson MLE driver in `src/mle.[ch]` with family-specific callbacks for the score, weight, and log-likelihood contributions. Same driver should extend to poisson, cloglog, etc. with no changes to the iteration logic — just new MleFamily structs.

Output matches Stata's layout: iteration log, LR χ^2, pseudo R^2, coefficient table with z-stats, optional cluster adjustment note.

Standard errors: - (default): classical (X'WX)^{-1} (asymptotic inverse-Hessian) - `vce(robust)`: HC1 sandwich - `vce(cluster v)`: CR1 cluster-robust on score

Numerical care: - log(1+e^η) computed stably as max(η,0) + log(1+e^{-|η|}) — no overflow. - Probit's φ/Φ uses asymptotic Mills bound λ(η) → -η when Φ(η) underflows. Per-obs Fisher weight clipped at 1e8. - Starting values: OLS solve, which also detects rank deficiency (collinear regressors marked omitted, held at 0 through iteration). - Step halving (up to 8 halvings) when full Newton step decreases □. - Perfect-separation detected when □ → 0; warning printed.

Verified: - Intercept-only on 50/50 sample: β_cons = 0 exactly. SE = 0.2 for logit, 0.1253 for probit (matches closed forms). - Real-effect data: classical β_logit/β_probit ≈ 1.81 ratio reproduced. - Newton converges in 3-5 iterations for well-conditioned data.

Postestimation: `test` and `lincom` work on logit/probit estimates. (test uses F with df_r = N-K; the asymptotic test would be χ^2, but the difference is negligible for typical sample sizes.)

`test` syntax extended (NEW)

The `test` command now supports the full Stata-style hypothesis syntax:

```
test var                        H0: β_var = 0
test var = 0                    same, explicit
test var = 0.5                  H0: β_var = 0.5 (numeric RHS)
test var1 = var2                H0: β_var1 = β_var2 (equality)
test var1 var2 ...              joint H0: all listed = 0
```

Internally constructs the restriction matrix R and value vector r, then computes the Wald F = (Rβ - r)' (R V R')^{-1} (Rβ - r) / q against F(q, df_r). Pre-existing `test L.growth L2.growth` (joint zero) is a special case and still works.

`lincom` accepts TS-op coefficient names (NEW)

The lincom parser now treats `.` as part of a coefficient name when followed by a name character, so `lincom L.growth + L2.growth` works correctly. Previously the parser stopped at the first `.` and tried to look up "L" as a coefficient.

`xtreg, re` — random-effects (GLS) estimator (NEW)

Implements panel RE via feasible GLS with quasi-demeaning:

```
xtreg y x1 x2 ... [if] [in], re [vce(robust|cluster v)]
```

Method: the model is `y_it = α_i + x_it'β + ε_it` with α_i ~ (0, σ_u²) assumed independent of x_it. The estimator transforms each variable by subtracting θ_i times the panel mean, where:

```
θ_i = 1 - σ_e / √(T_i · σ_u² + σ_e²)
```

Then runs OLS on (y, *X*, 1-θ_i) — note an explicit constant column, unlike FE. σ_e² comes from the within (FE) regression; σ_u² comes from the variance components computation (max(0, var(α̂_i) - σ_e²/T̄)). The transform interpolates between OLS (θ=0) and FE (θ=1).

Output matches Stata's xtreg, re layout: - "Random-effects GLS regression" header - R-within, R-between, R-overall — all computed with β_RE - σ_u, σ_e, ρ — variance components (shared with FE math) - θ — quasi-demeaning factor; min/avg/max for unbalanced panels -

_cons coefficient and SE (unlike FE) - Wald χ^2 for the slopes - corr(u_i, X) = 0 noted as the RE assumption

Verified on: - Degenerate hand-checkable case (perfect within fit → θ=1 → RE collapses to FE; β_RE = β_FE = 1.0 exactly; _cons omitted since the 1-θ column is identically zero). - WEO AR(1) growth: β_RE,L = 0.245 sits between pooled OLS and FE β = 0.220. σ_u/σ_e/ρ = 0.97/5.37/0.032 — most variance within- panel. θ varies 0.19-0.39 across 210 countries.

v1 limitations: - Uses $\sigma_u^2 = \max(0, \sigma^2_{BE} - \sigma_e^2/\bar{T})$ with $\bar{T} = N/n$ for unbalanced. Stata's exact Swamy-Arora uses a slightly different $\bar{T}$ correction; for balanced panels the estimators agree.

`hausman` — FE vs RE specification test (NEW)

```
xtreg y x1 x2 ..., fe
xtreg y x1 x2 ..., re
hausman
```

Computes H = (β_FE - β_RE)' [V_FE - V_RE]^{-1} (β_FE - β_RE), distributed χ^2(K_common) under H_0: cov(α_i, x_it) = 0. Rejection means RE is inconsistent; use FE.

Tea saves the last xtreg fe and last xtreg re into dedicated workspace slots automatically, so the user just types `hausman` with no arguments after running both. Order doesn't matter.

Implementation notes: - Only compares slope coefficients present (and not omitted) in both estimates. `_cons` is dropped since FE doesn't have one. - The difference matrix V_FE - V_RE should be PSD under H_0, but sampling noise (especially with robust/cluster SEs) can make it not strictly PD. Cholesky inverse is tried first; if it fails, falls back to an SVD pseudoinverse. When the resulting statistic comes out negative, we report |H| with a "sign flipped" warning. - Verified on WEO AR(1) growth: $\chi^2(1) = 189$, $p < 0.0001$ → strongly rejects RE (matches the FE-reported corr(u_i, Xb) = 0.97 warning sign). On a constructed case where α_i ⊥ x by construction, $\chi^2 \approx 0$ and the test correctly fails to reject.

`xtreg, fe` — fixed-effects (within) estimator (NEW)

Implements panel FE OLS via within transformation:

```
xtreg y x1 x2 ... [if] [in], fe [vce(robust|cluster v)]
```

Reports R-within, R-between, R-overall, σ_u, σ_e, ρ, corr(u_i, Xβ), and the F-test that all u_i = 0. Verified on a hand-checkable case (`tests/regression/29_xtreg_fe.do`): 3 panels × 4 obs, y = α_i + x with α ∈ {10,20,30} gives β_FE = 1.0 exactly, R-within = 1.0, σ_u = 10 exactly. Verified against the WEO AR(1) growth dynamic showing internally consistent results (β from `xtreg D.growth L.growth, fe` = -(1 - β from `xtreg growth L.growth, fe`)).

Standard error options: | `, fe` | classical: $\sigma^2 \cdot (X_w'X_w)^{-1}$ with df = N - n_groups - K | | `, fe vce(robust)` | HC1 sandwich on within-transformed data | | `, fe vce(cluster v)` | CR1 cluster-robust on within-transformed data |

v1 limitations (deferred to v2): - `_cons` is not displayed. Its point estimate (mean of α_i) is computable but the strictly correct SE requires LSDV-equivalent variance propagation we haven't implemented. Users wanting inference on _cons can run `regress y x i.panel` for now. - Stata defaults `vce(robust)` to cluster-by-panel for xtreg fe; tea's `vce(robust)` is HC1. Use explicit `vce(cluster panelvar)` for cluster-by-panel. - `xtreg, be` (between effects) is not yet implemented — returns a clean error suggesting `, fe` or `, re`. (`, re` ships in this version — see above.) - Weights work the same way as in `regress` (build_design handles them), but the FE-specific weight semantics have not been carefully verified yet.

TS-op handling unified across all varlist-accepting commands (NEW)

Time-series operator support is now centralized in `src/tsop.[ch]`. The parser handles every Stata form once (`L.x`, `L2.x`, `L(1/2).x`, `L.(x y)`, `L2.(x y)`, `L(1/2).(x y)`, step form `L(1(2)9).x`), and every command that accepts a varlist routes through `tsop_expand_varlist`. Concretely:

Command	TS ops	Mechanism
regress	☐	tsop_expand_varlist (depvar + xspec)
summarize	☐	tsop_expand_varlist
list	☐	tsop_expand_varlist
tabulate	☐	tsop_expand_varlist
tabstat	☐	tsop_expand_varlist
collapse	☐	tsop_expand_varlist (paren-aware tokenizer)

`regress` lost ~400 lines of duplicated parser code in this refactor (from 1198 down to 796 lines). Future commands that accept varlists — xtreg, ivregress, correlate, etc. — get TS-op support for free by calling `tsop_expand_varlist`. Test 28 exercises the universal coverage; tests 21–27 cover individual command surfaces.

Side fix discovered: `keep if` / `drop if` used to call `frame_unsort` which over-eagerly cleared `ts_panel` / `ts_time`. Row deletion doesn't invalidate panel structure (the columns are still there and still sorted; gaps are fine for tea's gap-aware tsidx lookup), so the ts state now survives those operations.

`xtdescribe` / `xtdes` (NEW)

Added `xtdescribe` (with `xtdes` abbreviation) to summarize the panel structure declared by `xtset`. Shows number of panels, time range, delta, span, min/p25/p50/p75/max distribution of obs per panel, and balanced/unbalanced determination. Test 27 exercises both balanced and unbalanced cases.

TS operators in `summarize` and `list` (FIXED)

`summarize L.x` and `summarize D.x` failed with "variable not found" because those commands called `varlist_expand` directly and didn't recognize TS-op tokens. The earlier TS-op fix only touched `regress`.

Fixed by adding a shared `tsop` module (`src/tsop.[ch]`). Both `summarize` and `list` now route their varlist through `tsop_expand_varlist`, which materializes TS-op tokens as temporary frame columns (with canonical display names like `L.x`, `D2.y`) and appends them to the frame for the duration of the command, then drops them via `tsop_drop_temps` before returning. Test 26 covers this.

D-operator iterated differencing (FIXED — silent math bug)

`D2.x`, `D3.x`, etc. all returned the same as `D.x` because the eval code hardcoded `back = -1` for any D-kind operator. This was a **silent** numerical bug: regressions including `D2.x` would compute correctly for `D.x` but the higher-order differences were all collinear with `D.x` and got "omitted" — looking like a model specification issue rather than a math error.

Stata's convention is that `D#.x` is the iterated `#`-th difference:

- `D.x  = x[t] - x[t-1]`
- `D2.x = D(D.x) = x[t] - 2·x[t-1] + x[t-2]`
- `D3.x = x[t] - 3·x[t-1] + 3·x[t-2] - x[t-3]`
- `D^k x[t] = Σ_{j=0..k} (-1)^j · C(k,j) · x[t-j]`

Fixed in `src/eval.c` using a binomial-recurrence loop. The `S` operator (separate semantics: simple gap-aware seasonal difference, `S#.x = x[t] - x[t-#]`) is unchanged. Test 25 verifies on `x = t²` where `D.x` is linear, `D².x` is constant 2, `D³.x` is constant 0.

Action item for users: any analysis using `D2.x`, `D3.x`, or higher in past sessions silently produced wrong values. Re-run any such code.

Operator-list TS syntax (FIXED)

Stata's compact form `regress y L(1/2).x` (expanding to `L1.x L2.x`) was rejected with "independent variables not found". Tea now handles the full set of Stata TS-op forms in regress varlists:

Syntax	Expands to
`L.x`	`L.x`
`L2.x`	`L2.x`
`L0.x`	`x` (no shift)
`L(1/3).x`	`L.x L2.x L3.x` (range)
`L(1 3).x`	`L.x L3.x` (explicit list)
`L(1(2)9).x`	`L.x L3.x L5.x L7.x L9.x` (step form)
`L.(x y)`	`L.x L.y`
`L2.(x y)`	`L2.x L2.y`
`L(1/2).(x y)`	`L.x L.y L2.x L2.y` (cross product)

Implemented in `src/regress.c` via the unified `try_tsop_form` parser plus `parse_numlist` (range / explicit list / step) and `find_matching_paren` for nested parens. The regressor tokenizer is paren-aware so spaces inside `(...)` don't break tokenization. Tests 21, 23, and 24 cover the forms.

Multi-digit time-series operators read wrong lag (FIXED)

`L2.x` was actually computing the 12th lag instead of the 2nd; `L3.x` the 13th, `F2.x` the 12th lead, `S3.x` a 13-period seasonal difference. Affected expressions, regress, and any other consumer.

The bug was in `try_tsop` in `src/parse.c`: the lag-magnitude accumulator was initialized to 1 instead of 0, so reading "L2" gave `num = 1*10 + 2 = 12`. Lag-1 (`L.x` / `L1.x` / no digits) accidentally worked because num stayed at 1 either way.

Fixed by initializing `num = 0`. The existing `if (!have) num = 1;` fallback still gives bare `L.` the right value of 1. Test 22 pins down correct multi-digit operator behavior.

Time-series operators rejected in regress varlist (FIXED)

`regress growth L.growth if country_id == "USA"` failed with "regress: independent variables not found" because the regress code called `varlist_expand` directly on each regressor token; that helper only knew about plain names, wildcards, ranges, and `_all` — not the `L.x` / `F.x` / `D.x` / `S.x` syntax.

Fixed by refactoring `build_design` in `src/regress.c` to detect TS-op tokens and route them through the existing expression evaluator (`expr_parse` → `N_TSOP`). Plain names still go through `varlist_expand`. Works for both the dependent variable and the regressors (`regress D.growth L.growth` is supported). Test 21 covers regress with TS operators.

capture and _rc (FIXED in v1.4.0)

Fixed: `capture` now suppresses all output of the captured command (stdout and stderr, at the file-descriptor level) and `_rc` returns the captured return code — 0 on success, the error code otherwise — exactly as in Stata. One small remaining deviation: a successful command that is NOT under `capture` leaves `_rc` unchanged rather than resetting it to 0; branch on `_rc` only immediately after a `capture`.

test 08 captures fewer lines than expected

The reshape-error regression test passes, but if you read the expected output you'll notice some error messages print to stderr while others to stdout, and the line ordering across the merged stream depends on line-buffering. This is OK as of v0.5.20 because we set the stdout buffer to line-buffered in `main.c`, but the harness assumes the ordering is stable — if a future change reorders error/output emission between commands, the test may need updating.

Storage types are display-only

`gen byte x = 1` accepts the `byte` qualifier but stores the value as double internally; `gen str10 country = "USA"` accepts `str10` but doesn't track width. This is a deliberate design decision documented in `COMPATIBILITY.md` — tea is double + string internally regardless of declared type, and the `.dta` writer auto-compresses on save. Not a bug.

Numeric function family coverage

Many Stata numeric functions are not yet implemented:

- Distribution functions: `normal`, `normalden`, `invnormal`, `chi2`, `chi2tail`, `invchi2`, `t`, `ttail`, `invttail`, `F`, `Ftail`, `binomial`, `binomialp`, `poisson`, `poissonp`.
- Matrix functions (we don't have a matrix type yet).
- Random-number generators: `runiform`, `rnormal`, etc. (Tied to `set seed`.)

The existing function set covers basic arithmetic, transcendentals, date/time, string manipulation, and the common aggregator functions used in `egen`.

v0.6 .dta polish (this milestone)

- **Value labels** — `label define` / `label values` sets are now written into `.dta` and read back via `use`. Round-trips cleanly (test 16).
- **Dataset label** — `label data "text"` now persists. Visible in `describe`, round-trips through save/use (test 17).
- **`save FILE.dta, version(NUM)`** — emit any DTA format 104-119 (test 18). Defaults to 118 (Stata 14).
- **Workspace label-set cleanup** — `clear` and `use, clear` now also drop the workspace's value-label sets, matching Stata semantics.

Still deferred from v0.6

- **Notes** — Stata's `note: text` on a dataset or variable. Not yet read or written.
- **strL strings > 2045 bytes** — currently strings longer than the str# limit are truncated at 2045 on write. Real-world risk is essentially zero for econ data; will add when needed.
- **`.dta` fweight metadata writes are silently dropped** — readstat 1.1.9 accepts `readstat_writer_set_fweight_variable` but does not serialize the resulting Stata `_fweight` characteristic. Tea's read-side captures the metadata correctly when present (so real Stata-saved files round-trip through tea correctly *if* the user doesn't save through tea), and the write-side wiring is in place for when readstat fixes this. Track upstream.

reported by user during real-data testing

- Bug 1 — _N in display context returned 1 → **FIXED in v0.5.20**
- Bug 2 — tabstat columns() ignored in by-mode → **FIXED in v0.5.19**
- Bug 3 — tabstat with if + by showed empty by-groups → **FIXED in v0.5.18**
- Bug 4 — list * segfaulted on 147-var dataset → **FIXED in v0.5.14**
- Bug 5 — quoted filename in import not opened → **FIXED in v0.5.12**

reported by Claude during synthetic testing

- Bug 6 — commas in string literals broke parser → **FIXED in v0.5.20**
- Bug 7 — `$1` in strings expanded as undefined macro → **FIXED in v0.5.20**
- Bug 8 — egen silently skipped strings → **FIXED in v0.5.20**

reported by Mico during WB fertility replication (v1.6.x)

- Bug 9 — `import excel, firstrow` header naming didn't match Stata: invalid chars were underscored (`Country Code` → `Country_Code`) instead of removed (`CountryCode`), and digit-leading headers (`1960`) got a `v` prefix instead of Stata's Excel column-letter fallback (`F`…`BQ`). Do-files written against Stata failed with `variable not found`. → **FIXED**: excel/ods import now uses Stata's exact rule; duplicate headers fall back to the column letter.
- Bug 10 — `foreach v of varlist BAD` was a **silent no-op**: an unknown variable made `varlist_expand` return −1, which the loop treated as an empty list. The body ran zero times with no error, and scripts failed much later with a misleading message. → **FIXED**: hard error, rc=111. The same −1-as-empty flaw was guarded at every remaining silent site: `by BAD:` (ran ungrouped!), `egen ..., by(BAD)` (ungrouped), `collapse, by(BAD)` (would collapse the whole dataset to one row), `format % BAD` (silent no-op). All now rc=111.
- Bug 11 — `local ++yr` parsed `++yr` as the macro *name*, silently defining a macro literally called `++yr` while `yr` never changed — loop counters froze and rename loops died with "already exists" on the second iteration. → **FIXED**: `local ++x` / `local --x` now increment/decrement per Stata; ++ on an undefined or non-numeric macro errors (rc=198) instead of silently minting a new macro.

reported by Mico during WB fertility replication, round 2 (v1.6.2)

- Bug 12 — `reshape long` silently **DROPPED every non-stub variable**: after `reshape long y, i(CountryCode IndicatorCode) j(year)` the carried columns (IndicatorName,

CountryName, ...) simply vanished, and the do-file died six lines later with a misleading `keep: variable not found`. → **FIXED**: carried variables are replicated across the j-rows of each wide observation, exactly as Stata does. Formats and variable labels survive on i() and carried columns.
- Bug 13 — `reshape` had a fixed 512-level cap that **silently dropped** level 513 onward (real WB pulls have ~1,900 indicator codes). → **FIXED**: dynamic sorted level arrays, binary-search everywhere.
- Bug 14 — `reshape wide` read j as numeric unconditionally, so `j(code_safe) string` was impossible. → **FIXED**: string j in both directions. New loud errors, never silent mangling: string j without the `string` option (rc 109), generated names that aren't valid identifiers (rc 198, with a strtoname() hint), missing/empty j values (rc 498), carried variables not constant within i() (rc 9), duplicate j within an i() group (rc 9 — previously last-write-wins, silently).
- Bug 15 — `merge` dropped variable labels (formats were kept). → **FIXED**: labels survive merge on master and using columns.
- Bug 16 — `frame_set_nobs` **leaked every string cell on shrink**: any row-deleting operation (`drop if`, `keep if`, `duplicates drop`) leaked the dropped rows' strings, and a later regrow overwrote the stale pointers, making it permanent. Invisible until reshape began carrying string columns into frames that then get row-pruned; caught by ASan. → **FIXED** in dataset.c.
- NEW — `preserve` / `restore` implemented (was on the roadmap): single depth like Stata, snapshot on disk as a native .tea tempfile so large frames don't double peak memory. `restore, not` discards; `restore, preserve` reloads but keeps the snapshot. A preserve still pending when a do-file concludes — normally or via abort — is restored automatically, with a note.
- NEW — `strtoname()` string function, Stata-exact: invalid characters become _, a leading digit gets a _ prefix, 32-char cap.

v1.6.3 — Stata-parity import naming + WASM refresh

- `import excel` WITHOUT `firstrow` now matches Stata exactly: columns are named by their Excel letters (`A`, `B`, ...) and row 1 is kept as a DATA row. (Previously tea always consumed row 1 as headers, silently losing the first data row.) Mixed columns (text header over numbers) become string columns, as in Stata.
- `import delimited` now applies Stata's naming rule: names are LOWERCASED by default and invalid characters are REMOVED ("Country Code" -> `countrycode`, not `Country_Code`); a header that is empty or starts with a digit falls back to the position name `v#`; duplicate headers also resolve to `v#`. New option `case(preserve|lower|upper)` for the Stata-compatible opt-out. NOTE: this is a deliberate behavior change — do-files that relied on tea's old underscored, case-preserving CSV names need `case(preserve)` or updated names. The bundled `sysuse` datasets are unaffected (already lowercase-clean).
- web/tea.wasm rebuilt — the browser engine is current again (it had been frozen at the v1.6.0 feature set while native moved ahead).
- Bug 17 — `tempfile` names were deterministic (`/tmp/tea_tmpN_name`), so any do-file using the tempfile + `file open, write` idiom **failed on its own second run** with rc=602 ("file already exists") — and leftover files accumulated in /tmp. → **FIXED**: paths now embed the pid (unique per session, like Stata's), and every tempfile handed out is deleted when tea exits.

v1.6.4 — quality-of-life extensions (not Stata, non-conflicting)

- NEW — `status`: one-line dataset summary — source filename, obs, vars, exact in-memory size, sort and xtset state. Source is tracked through use/import/sysuse, updated by save, cleared by clear (new Frame.source field, set at the command layer so internal frame operations like the reshape swap or restore never disturb it).
- NEW — progress indicator on long operations (import, reshape long, reshape wide, merge). Time-gated: nothing is drawn until ~1s of wall time, redraws at most every 250ms, stderr-and-TTY-only, erased on completion — so logs, pipes, the regression harness, `capture`, and short operations see nothing, ever. `set progress off` to disable. Compiled to no-ops in the WASM build (no TTY; a JS hook can come later).
- Bug 17 refinement: the tempfile uniqueness token is now pid ^ nanosecond-startup-time, because pid alone fails under emscripten (constant pid, and the node harness mounts the HOST /tmp) — caught by the WASM rig colliding with its own previous run.

reported by Mico during WEO test_04 (v1.6.5)

- Bug 18 — **`if COND {` evaluated against an empty scratch frame with `_N` hardwired to 1** — so `if _N > 0 {` was ALWAYS true and `if _N == 0 {` ALWAYS false, independent of the data. The diagnostic guard blocks in the WEO script ran unconditionally. Silently wrong control flow — the worst class. → **FIXED**: conditions evaluate against the active frame (_n=1, _N = real obs count); variable references like `x[1] == 5` now work in if-conditions too.
- Bug 19 — `forvalues i = 1/=_N'ran the body **ZERO times silently**: the range was parsed before macro expansion,sscanf' failed on the backtick, and 1/0 looped zero times with no error. → **FIXED**: the range is macro-expanded first, and an unparseable range is a loud rc=198 (a legitimately empty range like 1/0 still runs zero times, as in Stata).
- Bug 20 — compound quotes `"..."' were unsupported in macro expansion: the expander treated` as a macro reference and swallowed the text. → **FIXED** per Stata's rules: `" opens, "' closes, nesting tracked, macros still expand inside, a lone " inside is literal. display,`local x `"..."', and label variable accept them; plain "..." strings in display and label variable also accept the "" doubled-quote escape.
- Bug 21 — `label variable` stripped ALL trailing quotes from the label, mangling labels that contain quotes. → **FIXED** with a real quoted- string parser (see Bug 20).
- NEW — `quietly { ... }` / `capture { ... }` / `noisily { ... }` block forms (prefixes chain: `capture quietly { ... }`). `quietly` now also suppresses `display`, as in Stata (prefix and block forms both).
- NEW — extended macro function `local x : subinstr local|global y "from" "to" [, all]`, with compound-quoted arguments (the way to say a literal double quote). Other extended functions error loudly with the supported list rather than silently assigning the text.
- NEW — `error #` (abort with a return code; the do-file assertion idiom `if (bad) { ... error 459 }`) and `compress` (accepted for do-file compatibility; tea storage is already minimal — reports "(0 bytes saved)").
- NEW — `display as error|as text|as result|as input` and old-style `display in red|green|...` are recognized as style directives and no longer printed as literal text.
- Auto-restore of a pending `preserve` now also fires for do-files run from the command line (`tea script.do`), not just via the `do` command; preserve snapshots joined the atexit cleanup net.

v1.6.6 — graphics round two (the fertility-paper figures)

- NEW — **multi-series `twoway`**: parenthesized series each with their own `if` and options; plot types scatter / line / connected / lowess. Per-series `lcolor()` `lpattern()` `msymbol(i)` `mlabel()` `mlabcolor()` `mlabposition()` (clock); globals `yline(#,...)` (repeatable), `legend(off)` (simple legend otherwise), `yscale(range())`, `ylabel(a(s)b)` / `xlabel(a(s)b)`, `note()`. Axis titles inside a series merge upward, as in Stata. Tick rules and plot range are separate: a `ylabel()` rule never clips data.
- NEW — **`lowess`**: tricube kernel, running-line (default) or `mean`, `bwidth()` (0.8), `adjust`. Pure C, byte-identical across rigs.
- NEW — **`graph box`**: Stata percentile interpolation, adjacent-value whiskers (1.5 IQR), outside values (`noout` hides), one- or two-level `over()` (first varies fastest inside bands of the second), value labels honored, `relabel()`, `label(angle() labsize())`.
- NEW — **named-graph registry + `graph combine`**: `name(NAME[,replace])` stores the SVG in-session AND writes NAME.svg (documented deviation); `graph combine N1 N2, cols()/rows()` nests panels as scaled SVG; `graph dir` / `graph drop NAME|_all`. Name collision without `replace` is rc 110.
- Everything renders on the same deterministic SVG engine: %.2f coordinates with -0 snapped, golden-file diffed byte-identically on native, ASan, and WASM rigs (tests 56, 57). plot.c and the test-40 goldens are untouched.
- Documented deviations (COMPATIBILITY.md): name() writes a file; twoway line/lowess series are sorted by x; unknown cosmetic graphics suboptions are accepted-and-ignored (strict everywhere else).

v1.6.7 — value-label attachments survive frame rebuilds (Mico's fig1)

- Bug 22 — **encoded group variables silently reverted to raw numerics** in `graph box` band labels: reshape (all four carried-variable paths), merge (master and using), and `frame copy` copied format+vlabel but dropped `Variable.vallab`. All eight copy sites fixed.
- Bug 23 — the native format stored only the per-variable attachment NAME, not the label-set contents, so `save→clear→use` via `.tea` (and the preserve snapshot format) lost definitions. The format is now **TEA2**: value-label sets referenced by attached variables travel with the dataset, exactly like Stata's `.dta`. Legacy TEA1 files still read (without attachments); old tea versions cannot read TEA2.
- Bug 24 — `merge ... using FILE` defaulted a missing extension to `.tea` while use/save default to `.dta` — so savef'+merge … using `f'` on an extensionless tempfile failed with "cannot read ….tea". Merge now defaults to `.dta`, matching use/save and Stata.
- Test 58 locks all five survival paths: reshape wide+long, preserve/restore, `.tea` round-trip, `.dta` round-trip, and merge — asserted via rendered `graph box` band labels.

v1.6.8 — quality of life: conversion-phase progress for import excel

- `import excel` on a large workbook was silent for most of its wall time: the ssconvert/libreoffice child ran under a blocking system() with no feedback, and only the subsequent CSV-parse phase was byte-instrumented — so the progress line appeared "around 70%". The converter now runs under a polled fork/exec with a new progress ACTIVITY mode: spinner + elapsed seconds ("converting WB_data.xlsx / 34s"), same contract as the percentage line (stderr TTY only, 1s activation gate, 250ms redraws, erased on completion, `set progress off` disables, no-op on WASM). Batch output is byte-identical.

v1.6.9 — period-date constructors + eval errors abort (OECD HPI round)

- NEW — string period-date constructors: `quarterly(s,"YQ")`, `monthly(s,"YM")`, `halfyearly(s,"YH")`, `weekly(s,"YW")`, `yearly(s,"Y")`, and `daily(s,mask)` as the Stata alias of `date()`. Integer tokens are extracted left-to-right and assigned by the mask; two-digit years get +2000 (matching `date()`); an out-of-range period ("2020-Q7") yields missing, never a wrong date. The rest of the date family (mdy/dofq/yofd/qofd/... and %tq/%td/%tm display formats) already existed.
- Bug 25 — **a runtime evaluation error in gen/replace did not abort**: the error printed once per row, missing was stored anyway, each store counted as a "real change", and the command returned 0. A do-file with `gen qdate = quarterly(...)` under a build lacking the function marched on: collapse grouped on an all-missing year, merge matched ZERO rows, and the garbage panel was saved without a nonzero rc anywhere. Now: an eval error at the type probe fails before the variable exists; an error mid-loop aborts with rc=133 and rolls back completely — gen leaves no trace of the new variable, replace restores the column from a snapshot. Partial writes never survive.

v1.6.10 — `set more on|off` does something now

- `set more` had been accepted-and-ignored since v1.0 — a silent no-op: `set more on` armed nothing, and a stray `list` on a 400k-row panel flooded the terminal. Now it is Stata's output pager for real: with `set more on`, long `list` and `describe` output pauses at a screenful with `--more--`; space/any key = next page, Enter = one more line, q = stop with `--Break--`. Terminal height is read live (TIOCGWINSZ, fallback 24 rows). It engages ONLY in the interactive REPL with stdout a TTY — do-files, pipes, capture, logs, and the test suite never see it, so batch output is byte-identical. Default is off, matching modern Stata. `set more banana` errors (198), as does any other malformed argument — same strictness as `set progress`.

v1.6.11 — cellrange() + case() for import excel (WPP fertility round)

- NEW — `import excel ..., cellrange(A17:AF22000)`: restricts the import to a sheet rectangle. Real workbooks (the UN WPP file) carry title junk above the table; without cellrange, junk row 1 became the header. With `firstrow`, the range's first row is the header row. The end corner is optional (`cellrange(A17)` = from A17 to the sheet's end). The slice happens on the converted CSV with a fully CSV-aware state machine — quoted fields containing commas, doubled quotes, or embedded newlines slice correctly.
- NEW — `import delimited ..., rowrange(r1[:r2]) colrange(c1[:c2])`: Stata's rectangle restriction for text files, sharing the same slicer (and giving the regression suite a WASM-testable path — the xlsx conversion needs ssconvert, absent on that rig).
- Bug 26 — `case("lower")` with quotes, legal Stata syntax, ERRORED on import delimited and was **silently ignored** on import excel, which had no case() handling at all (names hardcoded to preserve). Both branches now accept quoted or bare arguments; the excel naming rule (invalid chars removed, column-letter fallback) folds case AFTER cleaning, and the excel default remains preserve, matching Stata.
- Test 60 locks range slicing (bounded + start-only), quoted-field survival, quoted case() args, and loud errors on malformed ranges.
- WASM-rig lesson #3 (joins Bug 17 and the test-57 note): emscripten NODEFS misbehaves after rename() — subsequent opens in the same directory can fail on stale

cache state. The slicer therefore never renames; it writes a `.rng` sibling that callers load and unlink. Slice temp names are per-call unique (pid ^ nanotime), not per-pid.

v1.6.12 — tabstat: value-labeled groups + format() (test_01_b round)

- Bug 27 — `tabstat ..., by(ctr_group)` named groups by raw numeric value (1/2/3) instead of the attached value labels (Advanced/Emerging/LIC). Same lookup rule as list and graph box now: labels when attached, `%g` fallback otherwise; string by-variables unchanged.
- Bug 28 — `format(%5.2f)` was ignored entirely — tabstat had no format() parsing and every cell went through a hardcoded %.7g. Now: the option applies to every cell INCLUDING the Total rows (the Total block carries a second copy of the cell-format macro, which the first fix pass missed — caught because the verification run showed a formatted table with an unformatted Total). Quoted format("%5.2f") accepted. Only printf-safe numeric formats pass validation (%w.d followed by f/g/e/F/G/E); anything else — format(%s), format(banana) — is a loud 198, since an arbitrary string reaching snprintf as a format is undefined behavior.
- Test 61 locks labeled groups, formatted group and Total rows in both table orientations, the quoted form, and both rejection paths.

v1.6.13 — long variable names no longer wreck sum/describe/tabstat

- Bug 29 — the WPP panel's 26-character names (totalpopulationasof1januar, ...) overflowed the fixed 12-char name column in `summarize` and the name columns/headers in `tabstat`, and shoved `describe`'s type/format columns out of alignment. Fixes: `summarize` and `tabstat` use Stata's abbrev() display rule (first n-2 characters +'~' + the last character: totalpopul~r), applied to variable names, column headers, and by-group labels alike; `describe` pads its name column dynamically to the longest name in view (16..32), header row included. Frames with short names render byte-identically to before — the whole existing suite passes against unchanged goldens.
- Test 62 locks the abbreviation rule in sum, tabstat headers and by-labels (including long VALUE labels), and describe's dynamic alignment.

v1.6.14 — documentation catch-up

- The manual and the generated command reference had lagged three releases of features: the import chapter now documents cellrange() (with the header-at-row-17 semantics), rowrange()/colrange(), quoted-and-bare case(); the date-function table gains the period constructors (quarterly/monthly/halfyearly/weekly/yearly and the daily alias); import's and tabstat's dispatch help strings (the source of the generated reference) now describe their full current option surface, including tabstat's format() and value-labeled by() groups. No behavior changes.

v1.6.15 — command echo for the browser editor (colleague feedback round)

- NEW — `set echo on|off`: echoes each do-file line Stata-style (". ") before its output. Default OFF — real Stata echoes `do` by default (`run` is the silent variant), but tea's batch-output contract and every golden test predate the feature; documented deviation. Interactive input never echoes (already visible). The browser editor's Run wraps the script in echo on/off, so the terminal now shows each command with its results beneath it.

- Browser editor hardening, from a colleague's "select-all + run deleted my script, leaving one stray character" report. The run shortcut itself was safe (preventDefault present); the real mechanism is a mistimed chord — Ctrl+A, then `d` landing after Ctrl is released, which replaces the whole selection with the letter d, and the 400ms autosave then persists the wreckage. Mitigations: the run shortcut now matches D case-insensitively (Shift/CapsLock used to defeat it and leak the key to the browser); every Run first snapshots the buffer to a backup slot; and a "restore last run" link swaps the backup in (reversibly). Native textarea undo (Ctrl+Z) also survives throughout.

v1.6.16 — estimation tables line up (the xtreg output round)

- Bug 30 — coefficient tables misaligned whenever a value needed more than its column: the cell format was %10.6g — six significant digits, MINIMUM width 10, no maximum — so -0.000367782 rendered 12 wide and sheared the row. All estimation tables (regress, xtreg fe/re/be, logit/probit, margins, hausman, arima — 19 sites) now format cells with gfit(), an emulation of Stata's %#.0g: the most significant digits that FIT the column, with the leading zero of |x|<1 dropped (-.0003678, .0000669) — exactly how Stata keeps coefficient columns rigid at any magnitude.
- The header stat blocks (xtreg's Number of obs / groups / R-squared / Obs per group / F / corr rows, and classic regress's ANOVA right column) now put every '=' in one fixed column with right-aligned values, Stata's layout.
- 22 goldens re-blessed; every changed line inspected — coefficient rows, header stats, and ANOVA cells only; no numeric drift (gfit reformats the same doubles).
- Table display precision is capped at SIX significant digits: the WASM rig immediately caught the 7th digit of longley's ill-conditioned CI bound differing between OpenBLAS and the reference BLAS (-5496.530 vs -5496.529) — the old 6-sig format had been hiding it. Displayed precision now equals the cross-rig reproducible bound; stored doubles keep full precision (e(b)/e(V) and exports are unaffected).

v1.6.17 — macOS build fails with words, not a garbage include path

- On macOS, a missing Homebrew dependency made `brew --prefix` return empty, the Makefile emitted `-I/include`, and the first thing the user saw was a mystifying "readstat.h file not found" deep into the compile. The Darwin block now checks every resolved prefix at parse time and fails immediately with the exact `brew install` command. Also: README's short macOS line was missing readline and readstat (the full line elsewhere was correct) — the likely root cause of the report. Linux/WASM builds untouched.

v1.6.17 — macOS build: honest errors for missing dependencies

- Bug 31 — on macOS with readstat not installed, `make` produced the mystifying `readstat.h file not found` behind a garbage `-I/include` flag. The Makefile's guard checked that `brew --prefix readstat` returned a non-empty string — but `brew --prefix` prints the would-be opt path even for a KNOWN-BUT-UNINSTALLED formula (exit 0), and comes back empty under other PATH/environment variants, so a non-empty prefix proved nothing. The guards now test for the actual header files (readstat.h, cblas.h, lapacke.h, gsl_cdf.h, readline.h) under each resolved prefix and fail with the exact brew command to run — including a from-source ReadStat recipe in case brew lacks the formula — plus a `make READSTAT_PREFIX=/path` override. Housekeeping targets (clean, distclean, showpaths) are exempt from the checks.

v1.6.18 — macOS: vendored ReadStat (there is no brew formula)

- Correction to v1.6.17's error message and README: **Homebrew has no readstat formula** (verified against homebrew-core) — `brew install readstat` was advice that cannot work, which is why the header guard fired with an empty prefix on a machine with everything else installed. New `make deps-readstat` target: clones WizardMac/ReadStat (–depth 1), builds it static into vendor/readstat, one command, ~a minute; the Makefile auto-detects the vendored copy afterwards on every platform and links libreadstat.a statically (one less runtime dependency). vendor/ is excluded from dist tarballs. macOS setup is now: brew install readline openblas lapack gsl && make deps-readstat && make. All 63 tests pass against ReadStat HEAD via the vendored path on the Linux rig.

v1.6.19 — make deps-readstat can actually run

- Bug 32 (chicken-and-egg, one release old): the v1.6.17 header guards fire at Makefile PARSE time, so on a machine without readstat, `make deps-readstat` — the cure the error message prescribes — died of the same error before its recipe could run. The exemption list (clean/distclean/showpaths) now also covers deps-readstat and checkdeps: the cure and the diagnosis must both work precisely when the dependency is missing. Verified from a pristine extracted tarball: deps-readstat → make → full test suite, no system readstat involved.

v1.6.20 — macOS: iconv is not free (the last mile of deps-readstat)

- Bug 33 — on macOS, `make deps-readstat` built libreadstat.a successfully and then died linking ReadStat's bundled extract_metadata tool: undefined _iconv/_iconv_open/_iconv_close for arm64. Root cause: iconv lives inside glibc on Linux but is a SEPARATE libiconv on macOS, and ReadStat's configure does not add it. Two-part fix: the deps-readstat recipe passes LIBS=-liconv to ReadStat's configure on Darwin, and — the failure one step further that this preempts — tea's own final link now appends -liconv (and -lz for readstat's compression paths) on Darwin, since the static libreadstat.a defers those symbols to the final link. -lz joined the common link line on all platforms (harmless where redundant); -liconv stays Darwin-only, as Linux glibc provides iconv natively and has no libiconv.so to link. Linux revalidated from scratch: vendor rebuild, clean build, 63/63.

v1.6.21 — clang-clean (Apple clang round, swept in one pass)

- Bug 34 — Apple clang's -Wunused-but-set-variable (an error under -Werror; Linux gcc does not flag these) stopped the macOS build at estcmd.c. Rather than fix one file per report, the whole tree was swept with clang-15 locally; three distinct issues, all fixed properly rather than suppressed: two genuinely dead counters in estimates dir/drop (deleted — Stata is silent there anyway), and the printf/fprintf log-tee macros redefining glibc's _FORTIFY_SOURCE macros without #undef (clang's -Wmacro-redefined).
- The Makefile is now compiler-aware: -Wno-format-truncation is GCC-only and upstream clang under -Werror rejects unknown warning options (Apple clang merely tolerates them) — the flag is applied only when CC is gcc.
- The full golden suite passes byte-identically on the clang-built binary — a fifth verification axis alongside gcc-native, ASan, WASM, and (pending Mico's machine) Apple silicon.

v1.6.22 — deep-clean under clang -Weverything

- Proactive sweep with clang-15 at -Weverything (minus deliberately pedantic families) to over-approximate any Apple clang generation. The tree came back almost clean; three findings, all fixed on merit: the `version` command embedded **DATE/TIME** — a compile timestamp that made builds non-reproducible byte-for-byte, gone (the version string carries release identity); one genuinely confusing dense one-liner in keep/drop reformatted (correct code, but -Wmisleading-indentation and any human reader agreed it looked wrong); the remaining family (-Wgnu-statement-expression in a macro) is a deliberate GNU-ism outside every compiler's default set.
- The golden suite passes byte-identically under the clang-built binary at the strictest flag set. If the macOS build still shows errors beyond v1.6.21's fixes, send `make 2>&1 | grep error: | sort -u` — the last paste arrived empty.

v1.6.23 — Bug 35: (long)missing is undefined behavior, and ARM proved it

- Mico's Apple-silicon run — the fourth rig — failed test 11: subinstr("aaa bbb aaa","aaa","X",.) returned the ORIGINAL string on macOS while replacing all on Linux. Root cause: the function evaluator cast raw argument doubles to long, and casting a MISSING (NaN-class) double is undefined behavior that diverges by ISA — x86's cvttsd2si yields LONG_MIN (negative, accidentally meaning "all"), arm64's fcvtzs yields 0 (limit zero: replace nothing). Same source, opposite results, no diagnostic anywhere. Notably the WASM rig never caught this: LLVM's wasm lowering happened to match x86.
- Every numeric argument coercion in the evaluator (30 sites) now checks missing FIRST and implements Stata's documented semantics: subinstr/subinword n=. means all occurrences; substr n1=. gives "", substr(s,n,.) gives the remainder; word(s,.)/char(.) give""; the entire date family propagates missing. The outputs are now ISA-independent by construction.
- Test 64 locks the defined semantics for all of it — when it passes on Apple silicon it is the ARM proof, since x86 accidentally produced most of these answers already.

v1.6.24 — Apple-silicon test-suite portability (58/64 -> 64/64 expected)

- Mico's first full suite run on Apple silicon: 58/64, test 64 GREEN — the ARM proof of Bug 35's fix. The six failures reduce to two causes, both suite-side, no engine changes:
- Five tests (56/57/58/60/63) printed `cd /tmp`'s output; macOS's /tmp is a symlink to /private/tmp and cd prints the resolved getcwd(). Those tests never meant to assert the path — `quietly cd /tmp` now, goldens re-blessed without the platform-dependent line.
- Test 41: longley's _se[year] differs in the SIXTH significant digit between x86 OpenBLAS and Apple-silicon OpenBLAS (.455479 vs .455478) — the near-singular matrix reproduces across BLAS implementations only to ~5-6 digits, one digit under the general display bound set in v1.6.16. Ruling: the general 6-digit table bound stays (it is right for well-conditioned problems); the TEST now smoke-checks longley with quietly regress + round()ed _b/_se/e() assertions at the precision this matrix actually reproduces on every rig. Longley keeps doing its job — probing near-singularity — without holding the byte-identity promise hostage to its own pathology.
- With these, all 64 goldens are expected green on all five platforms: x86 gcc, x86 clang, ASan/UBSan, WASM, and Apple silicon.

v1.6.25 — sysuse datasets carry variable labels

- The bundled datasets loaded with bare column names: they are embedded as raw CSVs, a format that cannot carry labels. Each dataset now embeds a companion variable-label table (data/NAME.lbl, plain "varlabel" lines; weo's 145 indicator labels generated from data/weo_codes.txt, truncated at the 80-char vlabel bound) which sysuse applies after the CSV load — describe on grunfeld now reads like the textbook, and the WEO extract documents itself.
- Fixed in passing: sysuse's missing-braces bug printed the "obs loaded" message even when the load failed (the if(rc==0) guarded only the source-string line) — the misleading-indentation class, caught by reading the code it sat next to.
- Test 46 re-blessed: outreg has ALWAYS preferred variable labels over names in export tables (publication behavior); it was simply unobservable until bundled data had labels. Test 65 locks labels across all six datasets.

v1.6.26 — native Excel import: the browser reads xlsx now

- NEW — src/xlsx.c: a native .xlsx reader (zip parsing + zlib inflate + sheet XML), replacing the ssconvert shell-out for .xlsx. No gnumeric or libreoffice needed anywhere, and — the point — `import excel` works IN THE BROWSER: drag a workbook into the page and import it. Handled: shared strings (incl. rich-text runs), CACHED FORMULA VALUES (a cell holding ='Sheet'!B2 imports its last evaluated value), inline strings, booleans, XML entities, gap cells, sheet() selection by name with a loud 601 for a missing sheet. Cells are emitted as CSV to the same temp path as before, so firstrow/case()/cellrange() behave identically. ssconvert remains the .ods path and an exotic- file fallback. The WASM build links an inflate-only zlib built from madler/zlib source.
- Bug 36 — the ssconvert-era cleanup rmdir'd the temp CSV's PARENT directory. Correct when that parent was the per-import /tmp/tea-xlsx-PID/ dir; with the native path writing straight into /tmp it became rmdir("/tmp") — silently a no-op on Linux (non-empty) but SUCCESSFUL on the browser's MEMFS once our own unlinks emptied it: /tmp deleted, every later temp operation (sysuse!) broken. The rmdir now fires only for directories named tea-xlsx*. Found because the demo rehearsal ran import-then-sysuse in one session — exactly the order a real workflow uses.
- Test 66 (with a stored-zip fixture in tests/fixtures/) locks the native reader on all rigs — including WASM, where import excel was previously impossible.

v1.6.27 — Bug 37: browser downloads silently cancelled

- "Download workspace files" counted the files and delivered none, and "Save .do" did nothing, in some browsers. Two independent defects with one visible symptom: the download anchor was never appended to the document (Firefox ignores programmatic clicks on detached anchors), and URL.revokeObjectURL() ran synchronously after click() — but click() only QUEUES the download, so the immediate revoke could destroy the blob before the browser read it. Whether anything landed depended on engine and timing; tea's own counter was honest ("3 new files downloaded") because the filesystem side succeeded — the browser then dropped the baton. Both handlers now share one triggerDownload(): append to DOM, click, remove, revoke deferred 10 s. Web UI only; the engine is unchanged.

v1.6.28 — Bug 38: multi-file download discarded by the permission prompt

- v1.6.27 fixed the detached-anchor and premature-revoke defects, and single-file downloads (Save .do) work. But "Download workspace files" with SEVERAL new files still delivered nothing: Chromium gates multiple programmatic downloads behind a permission prompt, and the attempts that TRIGGERED the prompt are discarded — "Allow" only covers future clicks. Diagnosed live on Edge the night before an IMF demo.
- Architectural fix: never fire more than one download. One new file downloads directly; several are bundled client-side into a single tea-workspace.zip (stored entries, CRC32, plain JS — ~50 lines, validated byte-for-byte against unzip). One file, one download, no permission prompt, every browser. Web UI only; engine unchanged.

v1.6.29 — the workspace download includes the data in memory

- Design gap, called by Mico: "Download workspace files" shipped only files the session had explicitly saved — but the workspace, as a user understands it, INCLUDES the dataset being worked on. sysuse + modify + download yielded nothing unless you remembered `save`.
- New quiet engine export tea_web_save_memory(path): serializes the current frame to .dta (format 118, labels and all) with no terminal output; returns 1 when memory is empty. The download handler calls it first and adds data_in_memory.dta to the bundle. Verified by round-trip: modified grunfeld (25 obs + generated variable + labels) and the demo's collapsed end-state both re-open natively, exact.
- The explicit-save workflow is unchanged; this only makes the download button's promise true.

v1.6.30 — save buttons with live state: memory is the deliverable

- Redesign, specified by Mico after demo rehearsals: the workspace-zip download conflated too much (script-saved intermediates, the editor autosave, charts) and had no notion of "unsaved". New semantics: "Save data (.dta)" downloads exactly the CURRENT IN-MEMORY dataset, and both save buttons carry live state — lit when there are unsaved changes, grayed otherwise.
- Engine: tea_web_data_hash(), an FNV-1a fingerprint of the frame (names, types, labels, all data bytes; 0 = empty). The UI re-hashes after every executed command and compares against the last-saved hash: reads (summarize, list) keep the button gray, writes (gen, drop, label, keep) light it, and a value-identical replace correctly stays gray — the fingerprint knows better than a dirty flag would.
- Editor: input events set a dirty flag; Save .do clears it. Charts: each plot in the Plots tab now carries its own "save NAME.svg" link (single-file downloads, no permission prompts).